Lab Exercise: Implementing Service Discovery with Eureka

Objective:

Implement two Spring Boot microservices that register with a central **Eureka Server**, allowing them to discover and communicate with each other.

Prerequisites:

- Java 17+
- Maven 3.8+
- Eclipse
- Basic knowledge of Spring Boot

Step 1: Create Eureka Server

1. Generate a new Spring Boot project using start.spring.io with the following dependencies:
    - Spring Web
    - Spring Cloud Discovery (Eureka Server)
2. Rename the project folder to `eureka-server`.
3. Open `EurekaServerApplication.java` and add the `@EnableEurekaServer` annotation:

```
package com.example.eurekaserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer // Enables Eureka Server functionality
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

4. Configure `application.yml`:

```
server:
      port: 8761 # Default port for Eureka dashboard
```

```yaml
eureka:
  instance:
    hostname: localhost
  client:
    register-with-eureka: false # Do not register self as client
    fetch-registry: false       # Do not fetch registry from others
```

5. Run the Eureka Server. Access the dashboard at:
   http://localhost:8761/

Step 2: Create First Microservice – `user-service`

1. Create a new Spring Boot project named `user-service` with:
   - Spring Web
   - Spring Cloud Discovery (Eureka Client)
2. Add the following code to `UserServiceApplication.java`:

```java
package com.example.userservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;

@SpringBootApplication
@EnableEurekaClient // Enable service registration
public class UserServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(UserServiceApplication.class, args);
    }
}
```

3. Configure `application.yml`:

```yaml
server:
  port: 8080

spring:
  application:
    name: user-service

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
```

4. Create a simple REST controller:

```java
package com.example.userservice.controller;
```

```java
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class UserController {

    @GetMapping("/users")
    public String getUsers() {
        return "User List from user-service";
    }
}
```

Step 3: Create Second Microservice – `order-service`

Repeat the same steps as above but name the application `order-service`, and use port `8081`. Also, create a similar controller:

```java
package com.example.orderservice.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class OrderController {

    @GetMapping("/orders")
    public String getOrders() {
        return "Order List from order-service";
    }
}
```

Update its `application.yml` accordingly:

```yaml
server:
  port: 8081

spring:
  application:
    name: order-service

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
```

Step 4: Inter-Service Communication Using Feign

Now, let's make `order-service` call `user-service` using **Feign Client**.

1. Add the **Spring Cloud OpenFeign** dependency in `order-service/pom.xml`:

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

2. Enable Feign Clients in `OrderServiceApplication.java`:

```
@EnableFeignClients
@SpringBootApplication
@EnableEurekaClient
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}
```

3. Create a Feign client interface:

```
package com.example.orderservice.client;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;

@FeignClient(name = "user-service") // Uses service discovery
public interface UserClient {
    @GetMapping("/users")
    String getUsers();
}
```

4. Update the `OrderController` to use the client:

```
package com.example.orderservice.controller;

import com.example.orderservice.client.UserClient;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class OrderController {

    @Autowired
    private UserClient userClient;

    @GetMapping("/orders")
    public String getOrders() {
        String users = userClient.getUsers(); // Calls user-service
```

```
        return "Order List from order-service | Users: " + users;
    }
}
```

Expected Output

After starting all services (`eureka-server`, `user-service`, and `order-service`), navigate to:

--> http://localhost:8081/orders

You should see output like:

`Order List from order-service | Users: User List from user-service`

This confirms that inter-service communication is working through Eureka-based service discovery .

Complete Implementation Files

Below are the full file structures for each module.

eureka-server

- `EurekaServerApplication.java`
- `application.yml`

user-service

- `UserServiceApplication.java`
- `UserController.java`
- `application.yml`

order-service

- `OrderServiceApplication.java`
- `OrderController.java`
- `UserClient.java`
- `pom.xml` (with Feign added)
- `application.yml`